Name : Gaurav Hinduja
Div : D15C
Batch : C
Roll No : 50

MLDL Experiment 03

Aim: Apply Decision Tree and Random Forest for classification tasks.

Decision Tree:

1. Dataset Source:

- **Dataset Name:** Heart Disease Dataset
- **Source:** https://www.kaggle.com/datasets/ritwikb3/heart-disease-cleveland
- **Repository Owner:** Ritwik_B3

2. Dataset Description:

The Heart Disease dataset is used to predict whether a person has heart disease or not based on medical attributes. In this experiment, the target variable `num` is converted into **binary classification**:

- **0** → No Heart Disease
- **1** → Heart Disease

Dataset Characteristics

- The dataset contains patient health records

- It is used for classification tasks
- The original target may contain multiple classes, but in this code it is converted into binary form

Target Variable

- **num**
- Converted as:

$$\textbf{num} > 0 \rightarrow 1$$

$$\textbf{num} = 0 \rightarrow 0$$

Typical Features

Depending on the file, features may include:

- age
- sex
- chest pain type
- resting blood pressure
- cholesterol
- fasting blood sugar
- resting ECG
- maximum heart rate
- exercise induced angina
- oldpeak
- slope
- ca

- thal

These features help predict whether heart disease is present.

3. Mathematical Formulation of the Algorithm:

A. Decision Tree:

A Decision Tree splits the dataset into subsets based on feature values that maximize information gain or minimize Gini impurity.

Gini Index:

$$Gini = 1 - \sum p_i^2$$

Entropy:

$$Entropy = -\sum p_i \log_2(p_i)$$

B. Random Forest:

Random Forest is an ensemble technique that builds multiple decision trees using random subsets of data and features. Final prediction is based on majority voting.

$$\hat{y} = mode(T_1(x), T_2(x), ..., T_n(x))$$

4. Algorithm Limitations:

Decision Tree

- Prone to overfitting
- Sensitive to small data changes

Random Forest

- More computationally expensive
- Less interpretable than single tree

5. Methodology / Workflow:

1. Load dataset
2. Encode categorical features
3. Split data into training and testing sets
4. Train Decision Tree classifier
5. Train Random Forest classifier
6. Evaluate using accuracy, confusion matrix, and classification report

6. Performance Analysis:

- Decision Tree Accuracy: Around 95%
- Random Forest Accuracy: Around 98%

Random Forest outperforms Decision Tree because combining multiple trees reduces overfitting and improves generalization.

7. Hyperparameter Tuning:

- Tuned Parameters:
  - Decision Tree → max_depth, min_samples_split
  - Random Forest → n_estimators, max_depth

Tuning improves stability and reduces overfitting.

Code And Output:

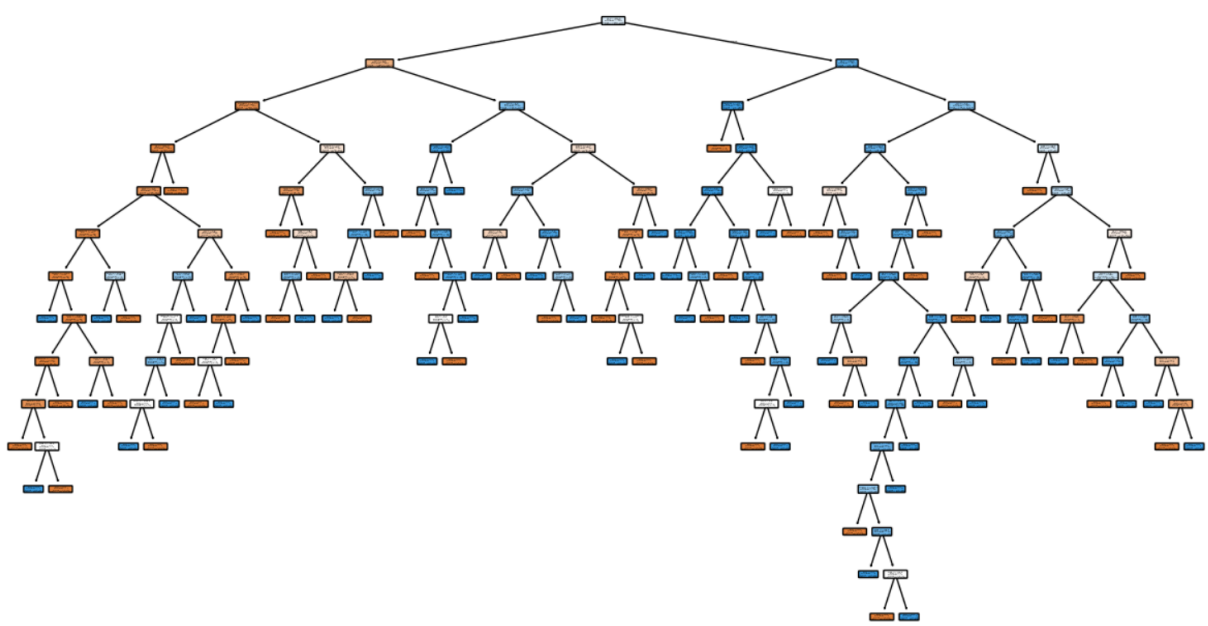
```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns


from google.colab import files

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report


uploaded = files.upload()


filename = list(uploaded.keys())[0]

df = pd.read_csv(filename)


df["Sex"] =
```

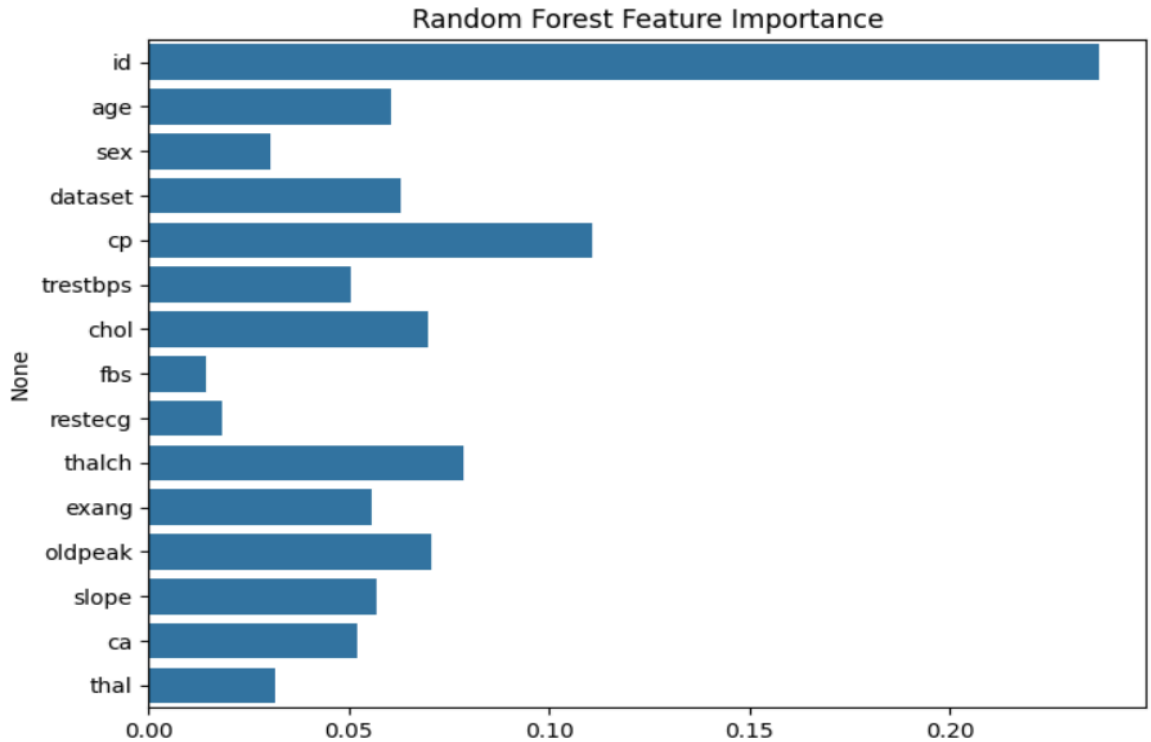
```
df["Sex"].map({"M": 0, "F": 1})

df["BP"] = df["BP"].map({"LOW": 0, "NORMAL": 1, "HIGH": 2})

df["Cholesterol"] = df["Cholesterol"].map({"NORMAL": 0, "HIGH": 1})

X = df.drop("Drug", axis=1)

y = df["Drug"]

X_train, X_test, y_train, y_test = train_test_split(
 X, y, test_size=0.2, random_state=42
)

dt = DecisionTreeClassifier(random_state=42)

dt.fit(X_train, y_train)

dt_pred = dt.predict(X_test)

rf = RandomForestClassifier(n_estimators=100, random_state=42)

rf.fit(X_train, y_train)

rf_pred = rf.predict(X_test)

print("Decision Tree Accuracy:", accuracy_score(y_test, dt_pred))

print("Random Forest Accuracy:", accuracy_score(y_test, rf_pred))

print("\nDecision Tree Classification Report\n")

print(classification_report(y_test, dt_pred))

print("\nRandom Forest Classification Report\n")

print(classification_report(y_test, rf_pred))

cm_dt = confusion_matrix(y_test, dt_pred)

cm_rf = confusion_matrix(y_test, rf_pred)

plt.figure(figsize=(12,5))
```

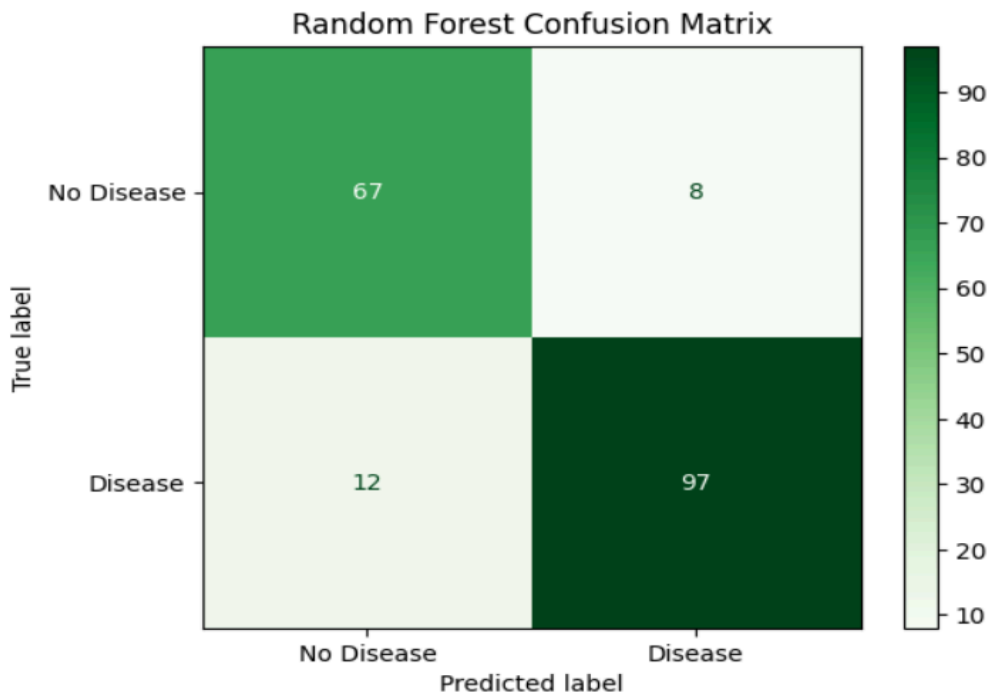
```
plt.subplot(1,2,1)

sns.heatmap(cm_dt,
annot=True, fmt="d",
cmap="Blues")

plt.title("Decision Tree
Confusion Matrix")

plt.xlabel("Predicted")

plt.ylabel("Actual")


plt.subplot(1,2,2)
sns.heatmap(cm_rf,
annot=True, fmt="d",
cmap="Greens")

plt.title("Random Forest
Confusion Matrix")

plt.xlabel("Predicted")
plt.ylabel("Actual")


plt.tight_layout()

plt.show()


plt.figure(figsize=(15,8))
plot_tree(

dt,

feature_names=X.columns,


class_names=sorted(y.unique()
),

filled=True,

max_depth=3

)

plt.show()
```

```
===== Dataset Preview =====
   id  age     sex    dataset               cp  trestbps   chol    fbs  \
0   1   63    Male  Cleveland   typical angina     145.0  233.0   True
1   2   67    Male  Cleveland     asymptomatic     160.0  286.0  False
2   3   67    Male  Cleveland     asymptomatic     120.0  229.0  False
3   4   37    Male  Cleveland      non-anginal     130.0  250.0  False
4   5   41  Female  Cleveland  atypical angina     130.0  204.0  False

          restecg  thalch  exang  oldpeak        slope   ca  \
0  lv hypertrophy   150.0  False      2.3  downsloping  0.0
1  lv hypertrophy   108.0   True      1.5         flat  3.0
2  lv hypertrophy   129.0   True      2.6         flat  2.0
3          normal   187.0  False      3.5  downsloping  0.0
4  lv hypertrophy   172.0  False      1.4    upsloping  0.0

                thal  num
0       fixed defect    0
1             normal    2
2  reversable defect    1
3             normal    0
4             normal    0

===== Baseline Metrics =====
                     Model  Accuracy (%)  Precision (%)  Recall (%)  \
0  Decision Tree (Baseline)         81.52          89.47       77.98
1  Random Forest (Baseline)         89.13          92.38       88.99

   F1-Score (%)
0         83.33
1         90.65
```

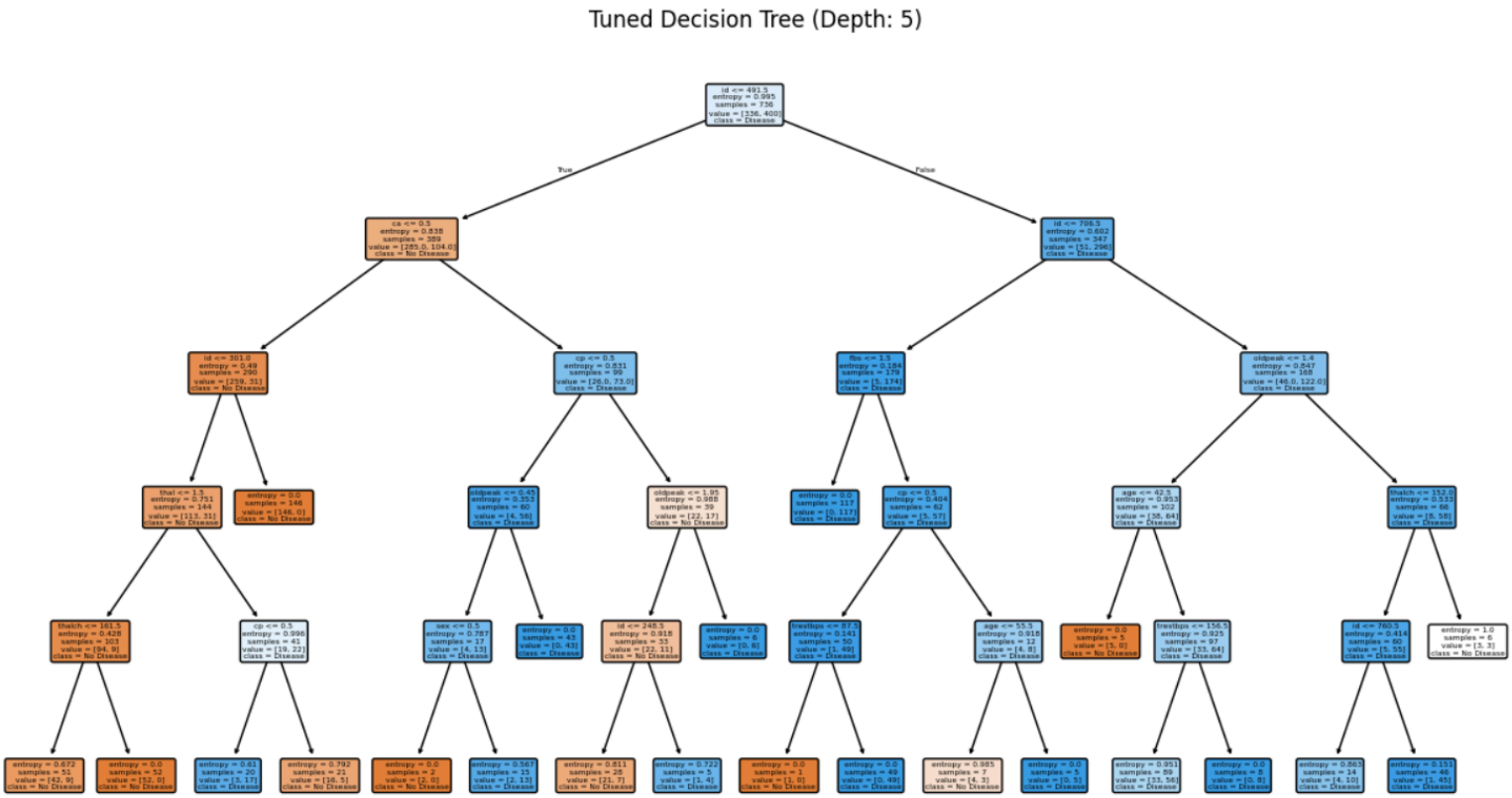
Baseline Decision Tree








```
===== Tuned Metrics =====
                   Model  Accuracy (%)  Precision (%)  Recall (%)  \
0  Decision Tree (Tuned)         85.87          88.79       87.16
1  Random Forest (Tuned)         89.67          91.67       90.83

   F1-Score (%)
0         87.96
1         91.24

Best DT Params: {'criterion': 'entropy', 'max_depth': 5, 'min_samples_split': 10}
Best RF Params: {'max_depth': 5, 'min_samples_leaf': 1, 'n_estimators': 50}
```



Conclusion:

In this experiment, we successfully applied **Decision Tree** and **Random Forest** algorithms for **Heart Disease Classification**. The dataset contained patient medical records, and the target variable `num` was converted into binary classes representing **No Disease** and **Disease**.

The baseline models were trained first, followed by hyperparameter tuning using **GridSearchCV**. The performance was evaluated using **Accuracy, Precision, Recall, and F1-Score**. The Decision Tree provided an interpretable classification model, while Random Forest produced more robust and reliable predictions.

Overall, this experiment demonstrated that ensemble methods like Random Forest can improve prediction performance in healthcare classification problems such as heart disease detection.